

Trabajo Integrador - IA en Producción

[Trabajo Integrador - IA en Producción](#)

[Descripción](#)

[Contexto / Problema](#)

[Objetivos del Desarrollo](#)

[Arquitectura](#)

[Usuarios y Casos de Uso](#)

[Datasets a Utilizar](#)

[Definición de la API REST para Consulta](#)

[Formato de Entrega](#)

[Requerimientos entrega parcial \(16/4\)](#)

[Requerimientos entrega final \(28/5\)](#)

[Evaluación](#)

[Aclaraciones](#)

Descripción

Este documento describe los requerimientos para el desarrollo de una plataforma que permita pronosticar la producción futura de hidrocarburos.

El producto busca mejorar la previsibilidad del volumen producido y reducir la incertidumbre en la planificación operativa mediante una plataforma que permita a los equipos técnicos y de planificación optimizar la toma de decisiones y anticipar escenarios de producción.

El objetivo del trabajo es el desarrollo de un pipeline para puesta en producción de un modelo de ML.

El sistema incluirá:

- Módulo de carga e integración de datos históricos de producción, pozos y variables operativas.
- Motor de modelado y pronóstico basado en algoritmos estadísticos y/o de machine learning.
- API REST para consulta, integración y consumo de resultados desde sistemas externos.
- Registro y trazabilidad de cambios en modelos y supuestos.

El desarrollo va a ser llevado a cabo en grupo de 2 alumnos. Cada equipo deberá disponibilizar al equipo docente un repositorio de código privado donde se llevará a cabo el desarrollo.

Notas:

- El foco del trabajo está en los procesos de ML Engineering, no en la precisión o sofisticación de los modelos de predicción.
- En este documento se utilizará la convención del [RFC 2119](#).

Contexto / Problema

Los equipos de planificación, ingeniería de reservorios y operaciones enfrentan dificultades para estimar con precisión la producción futura de hidrocarburos, lo que genera:

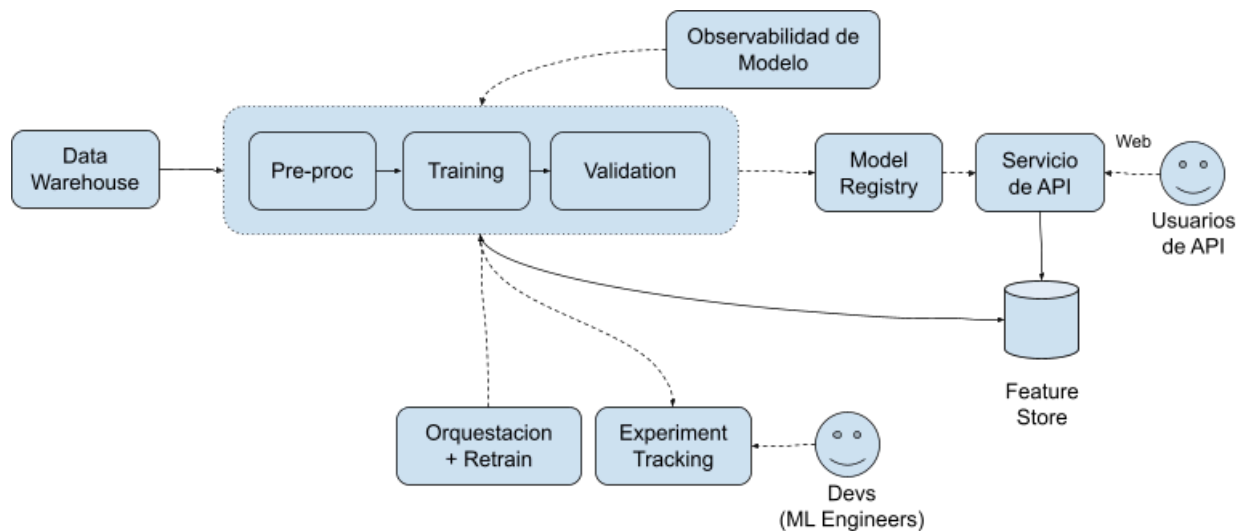
- Alta incertidumbre en la planificación operativa y presupuestaria.
- Decisiones reactivas basadas en información incompleta o desactualizada.
- Pérdida de oportunidades de optimización en pozos y activos.
- Dificultades para planificar inversiones, mantenimiento y compromisos comerciales.

Actualmente los pronósticos se realizan mediante planillas dispersas, modelos manuales o herramientas no integradas, con fuerte dependencia del conocimiento individual y limitada trazabilidad sobre los supuestos utilizados.

Objetivos del Desarrollo

- **Pronóstico:** Sistema que pueda proveer una estimación anticipada de la producción de hidrocarburos a corto, medio y largo plazo.
- **Reducción de Incertidumbre:** Minimizar la incertidumbre en la planificación operativa y presupuestaria.
- **Integración Sistémica:** Habilitar la consulta y el consumo automático de pronósticos vía API REST por otros sistemas corporativos.
- **Trazabilidad de Modelos:** Garantizar la auditabilidad y el linaje de los modelos predictivos y sus supuestos.

Arquitectura



Usuarios y Casos de Uso

- Usuarios de API
 - **DEBEN** poder consumir una API REST que permita hacer uso del servicio.
- Devs (ML Engineers)
 - **DEBEN** poder acceder a una plataforma de tracking de experimentos de machine learning.

Datasets a Utilizar

- Producción de Pozos de Gas y Petróleo No Convencional:

https://datos.gob.ar/dataset/energia-produccion-petroleo-gas-por-pozo-capitulo-iv/archivo/energia_b5b58cdc-9e07-41f9-b392-fb9ec68b0725
- Listado de pozos cargados por empresas operadoras (informacion complementaria):

https://datos.gob.ar/dataset/energia-produccion-petroleo-gas-por-pozo-capitulo-iv/archivo/energia_cbfa4d79-ffb3-4096-bab5-eb0dde9a8385

Definición de la API REST para Consulta

El sistema **DEBE** exponer una API RESTful para el acceso programático a los resultados del pronóstico (**Requerimiento Funcional: API de Consulta (REST)**).

- **Endpoints:**
 - **GET /api/v1/forecast:** Obtiene el pronóstico base para un horizonte de

tiempo y nivel de desagregación (ej. activo, yacimiento).

- Parámetros de consulta: `id_well` (identificador del pozo), `date_start`, `date_end`.
- Campos de respuesta: `id_well` (identificador del pozo), array de objetos json con la producción esperada para cada fecha entre `date_start` y `date_end`. Cada elemento del array tiene estos campos: `date` (fecha), `prod` (volumen producido).
- `GET /api/v1/wells`: Obtiene el listado de pozos.
 - Parámetros de consulta: `date_query` (fecha para la cual se quiere hacer la consulta).
- **Formato de Datos**: JSON estándar.
- **Documentación**: La API **DEBE** seguir la especificación definida en formato OpenAPI (Swagger).

Especificación en formato OpenAPI que se utilizará para validar el funcionamiento del servicio desarrollado (sujeto a cambios futuros):

```
openapi: 3.0.3
info:
  title: Oil & Gas Forecast API
  version: 1.0.0
  description: API simple para consultar el listado de pozos y sus pronósticos de producción.

paths:
  /api/v1/forecast:
    get:
      summary: Obtiene el pronóstico de producción de un pozo.
      parameters:
        - name: id_well
          in: query
          required: true
          description: Identificador del pozo.
          schema:
            type: string
        - name: date_start
          in: query
          required: true
          description: Fecha de inicio (YYYY-MM-DD).
          schema:
            type: string
            format: date
        - name: date_end
          in: query
          required: true
          description: Fecha de fin (YYYY-MM-DD).
          schema:
```

```
    type: string
    format: date
responses:
  '200':
    description: Pronóstico obtenido exitosamente.
    content:
      application/json:
        schema:
          type: object
          properties:
            id_well:
              type: string
              example: "POZO-001"
            data:
              type: array
              description: Arreglo con la producción esperada.
              items:
                type: object
                properties:
                  date:
                    type: string
                    format: date
                    example: "2023-10-01"
                  prod:
                    type: number
                    format: float
                    example: 150.5

/api/v1/wells:
  get:
    summary: Obtiene el listado de pozos.
    parameters:
      - name: date_query
        in: query
        required: true
        description: Fecha para la cual se hace la consulta (YYYY-MM-DD).
        schema:
          type: string
          format: date
    responses:
      '200':
        description: Listado de pozos obtenido exitosamente.
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  id_well:
                    type: string
```

example: "POZO-001"

Formato de Entrega

Cada entrega estará compuesta por los siguientes ítems:

- Commit al repositorio de git que se tomará para la entrega. Este commit deberíamos encontrar:
 - Implementación del proyecto incluyendo código, configuraciones y demás archivos para levantar la solución.
 - README actualizado con:
 - Instrucciones para acceder y hacer uso de todos los componentes del desarrollo.
 - Aspectos de diseño relevantes del proyecto.
- Video de 6 minutos a modo de demo del desarrollo realizado, con participación de todos los miembros del equipo.

Requerimientos entrega parcial (16/4)

- El sistema **DEBE** poder levantarse localmente mediante docker-compose.
- El sistema **DEBE** exponer una API funcional conforme con la especificación OpenAPI presentada.
- Se **DEBE** incluir una funcionalidad que permita realizar tracking del entrenamiento de modelos de modo que el entrenamiento del mismo sea reproducible y se pueda saber que modelo está productivo.
- Se **DEBEN** loguear métricas y artefactos relevantes en al entrenar el modelo para entender su performance.
- La generación de features a partir de los datos crudos **DEBE** quedar persistido en un feature store que será utilizado durante la inferencia.
- El entrenamiento del modelo **DEBE** llevarse a cabo consumiendo del feature store.
- **DEBE** ser posible llevar a cabo el entrenamiento del modelo de manera repetible con un solo comando para cualquier día dado que indique el usuario.

Requerimientos entrega final (28/5)

- Todos los requerimientos de la primer entrega
- El sistema **DEBE** llevar a cabo el entrenamiento y despliegue de modelos de manera recurrente y automática mediante alguna herramienta de orquestación (ej Airflow). La frecuencia de updates estará dada por el dataset.

- El sistema **DEBE** dar un reporte de model decay, data drift / concept drift con al menos dos métricas que nos permitan observar cuando la performance del modelo se aleja de la esperada.
- El sistema **DEBE** implementar una arquitectura escalable para responder la inferencia de la API (ej: Ray).

Evaluación

Con el fin de evaluar cada entrega, se considerarán los siguientes aspectos:

- Cumplimiento de los requisitos definidos en cada adenda técnica en tiempo y forma.
- Capacidad del equipo para responder preguntas sobre el diseño y funcionamiento del sistema incluyendo: supuestos y limitaciones considerados, trade-offs de las alternativas analizadas y la justificación técnica de las soluciones desarrolladas.
- La rúbrica de evaluación se desprende directamente de los requerimientos de la entrega final.

Aclaraciones

- Se requiere utilizar *pull requests* (PRs) de manera prolija cuando se trabaje colaborativamente a fin de ejercitar prácticas usuales de desarrollo en equipo, al mismo tiempo que facilite la contribución de cada miembro del equipo.
- **Se espera que la historia de *commits* y PRs refleje el trabajo distribuido entre las distintas personas del equipo sin excepción.**
- No se pide que el sistema desarrollado corra en un entorno cloud pero tampoco está prohibido en caso de que los miembros del equipo así lo prefieran.